

🔒 ТЕХНИЧЕСКИЙ ПАСПОРТ И ИБ-АНАЛИЗ КОНТУРА ОФИЦЕРА БЕЗОПАСНОСТИ (security_rights.js)

Настоящий архитектурный документ содержит исчерпывающее описание, пошаговый структурный разбор и профиль информационной безопасности специализированного фронтенд-контура, предназначенного для изоляции действий Менеджеров безопасности (security_rights.js).

Все алгоритмы распила JWT-токенов, веерного наполнения селекторов, проверки кадрового совпадения логинов и санитизации полей переведены в пуленепробиваемые HTML-модели таблиц. Это гарантирует стопроцентное сохранение пропорций, выравнивания и геометрии элементов на страницах итогового PDF-документа.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур обеспечивает запуск механизма вычитки DSN серверов, снайперское извлечение Payload активной сессии оператора из заголовков JWT и защиту от повторного входа.

--- Схема декомпрессии Payload JWT и блокировки гонки селекторов ---

ВЫЗОВ СЛУЖБЫ: securityLoadServersForSelect()

⬇ Затвор гонки: `if (window.isIbSelectorLoading)` —► Мгновенный откат

ЭТАП А: ПЕРЕХВАТ И ДЕКОДИРОВАНИЕ СЕССИОННОГО КЛЮЧА ИБ

`localStorage.getItem('pg_access_token')` —► `sessionToken.split('.')`

⬇ Криптографический разбор полезной нагрузки в ОЗУ

**Контур А: Декодирование успешно
пройено**

Контур Б: JWT поврежден / отсутствует

● Вызов: `atob(base64) + decodeURIComponent()`.
📄 Чтение полей: `tokenData.sub || tokenData.username`.
💾 Фиксация: `cleanTargetUser =` логин оператора.

⚠ Активация резервного шага страховки.
📄 `localStorage.getItem('username')`.
✂ Санитизация: Срезание кавычек через `regex.replace(/^[\"']|[\"']$/g, '')`.

ПЕРЕХОД К ЭТАПУ СВЕРКИ КАДРОВОГО СОВПАДЕНИЯ СЕССИЙ

🛡 Профиль ИБ-защиты подсистемы инициализации ИБ-контура:

- **Защита от атак повторного запроса и перегрузки ОЗУ (Race Condition):** Атомарный маркер `window.isIbSelectorLoading = true` на строке 19 полностью блокирует отправку повторных

или параллельных запросов выгрузки списков серверов, если предыдущая транзакция еще не завершила рендеринг DOM-дерева в браузере.

- **Многоуровневая верификация личности оператора (Anti-Spoofing):** Извлечение логина выполняется снайперским распилом токена `sessionToken.split('.')` и нативным парсингом payload. Текстовые переменные локального кэша используются исключительно как резервный фолбек при недоступности JWT, при этом проходя обязательную санитизацию регулярным выражением от кавычек, ликвидируя риск инъекций.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур осуществляет фильтрацию глобального реестра серверов на основе принципа жесткой кадровой изоляции и собирает уникальный массив допусков для текущей сессии.

--- Схема кадровой фильтрации и сборки допусков хостов ---

РЕЕСТР ИЗ API: `const userRes = await secureFetch('/api/admin/panel-users-list')`

↓ Считывание числового ID: `mySessionUserId = Number(myProfileObj.id)`

КОНТУР СТРОГОГО КАДРОВОГО ФИЛЬТРА ОПЕРАТОРА ПАНЕЛИ

`usersList.filter(u => u.username.toLowerCase() === cleanTargetUser)`

↓ Разветвление рантайм-логики валидатора лимитов

**Уникальный Set() хостов пуст
(`uniqueOptions.size == 0`)**

- Полная блокировка контура управления ИБ.
- 💬 Написание опции: 'У вашего профиля нет разрешенных серверов'.
- 🔒 Навешивание принудительного атрибута `setAttribute('disabled', 'true')`.

Обнаружены легитимные привязки серверов

- 📄 Алфавитная сортировка массива хостов: `.sort()`.
- ✂ Генерация чистых HTML-тегов `option value="${finalServerName}"`.
- 🔒 Атомарный снос блокировок: `removeAttribute('disabled')`.

ФИНАЛ: ВЕЕРНАЯ ИНЪЕКЦИЯ ДАННЫХ В ОБА СЕЛЕКТОРА ЭКРАНА ЗАВЕРШЕНА

🛡 Профиль ИБ-защиты подсистемы кадровой изоляции:

- **Защита от межпользовательской компрометации (Horizontal Privilege Escalation):** Фильтр на строках 72–78 жестко отсекает любые чужие записи лимитов из памяти. Инструкция `return dbUser === cleanTargetUser` гарантирует, что ИБ-офицер получит доступ строго к тем серверам

кластера, которые были явно привязаны к его учетной записи Суперадминистратором, блокируя несанкционированный просмотр структуры чужих хостов.

- **Предотвращение инъекций мусорных байт:** При итерации по массиву `srvData` внедрен жесткий затвор `s !== 'all' && s !== 'none_readonly' && s !== '[object object]'`. Это блокирует попытки вывести системные маркеры безлимита или текстовые ошибки рендеринга в выпадающий список селектора, обеспечивая абсолютную чистоту DOM-инпутов.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур обеспечивает двухкаскадную асинхронную подгрузку доступных баз данных и ролей СУБД для выбранного инстанса, блокируя отображение неразрешенных объектов.

--- Схема каскадного наполнения баз данных и сканирования ролей ---

ТРИГГЕР ИЗМЕНЕНИЯ СЕРВЕРА (securityOnServerChanged)

↓ Запрос баз: `secureFetch(`/api/get-target-databases/${serverId}`)`

ЭТАП А: НАПОЛНЕНИЕ СЕЛЕКТОРА БАЗ И ПЕРЕХОД К НАЖАТИЮ (securityOnDatabaseChanged)

↓ Параллельный опрос системных/кастомных групп: `Promise.all()`

Поток 1: Кастомные роли (show_system=false)

- Вычитка массива ролей из `dataCustom.roles`.
- 📦 Прямое добавление в уникальное ОЗУ-множество `uniqueUsers.add()`.

Поток 2: Системные группы (show_system=true)

- Вычитка системных префиксов `pg_*`.
- 🔍 Сверка с лимитами Офицера: `panel_user_allowed_roles`.
- ✗ Отсечение несанкционированных групп.

ФИНАЛ: СОРТИРОВКА И ВЫВОД РАЗРЕШЕННЫХ ПОЛЬЗОВАТЕЛЕЙ В СЕЛЕКТОР

🔵 Профиль ИБ-защиты подсистемы каскадного сканирования:

- **Мгновенная изоляция интерфейса при смене фокуса (UI-Reset):** При каждом изменении выбранного сервера или базы данных, элемент управления `secInlinePanel` принудительно скрывается инструкцией `display: none !important`. Это исключает риск проведения операций с устаревшим контекстом (например, выдачу прав на сервере Б, когда на экране всё ещё отображаются параметры сервера А).
- **Снайперский контроль системных групп доступа СУБД:** При разборе системных ролей алгоритм сверяет каждую позицию с локальным кэшем разрешенных ролей оператора. ИБ-офицеру разрешено увидеть и выбрать только ту группу (например, `pg_checkpoint`), которая

была явно делегирована ему администратором панели, исключая горизонтальный перехват контроля.

- **Превентивный инъект суперпользователя (Root Guard):** В строку 226 вшит принудительный инъект `uniqueUsers.add('postgres')`. Это гарантирует, что корневой аккаунт СУБД всегда доступен для экстренных манипуляций со стороны Менеджера безопасности.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур обеспечивает реактивное извлечение и сопоставление веерного пула разрешенных ролей текущего оператора панели на основе перехвата его идентификатора `user_id`.

--- Схема динамического сбора разрешенных ролей оператора из RAM ---

ВЫБОР ЦЕЛЕВОЙ РОЛИ СУБД: `securityOnUserSelected()`

↓ Декодирование логики оператора из JWT по ключу `sub / username`

ЭТАП А: СОПОСТАВЛЕНИЕ ИМЕНИ С ЧИСЛОВЫМ `USER_ID` В СУБД

`myProfileObj = usersList.find(...)` —> Изоляция строк строго по `user_id`

↓ Агрегация лимитов: `const myAllowedRoles = activeRoleForDisplay.split(',')`

В базе отсутствуют лимиты ролей оператора

Пул ролей успешно сгруппирован в ОЗУ

● Полная блокировка инлайн-выпадающего списка.

💬 Инъект: 'У вашего профиля нет разрешённых ролей...'

🔒 Отключение главной кнопки `submitBtn.setAttribute('disabled', 'true')`.

abc Алфавитная сортировка уникального Set-массива.

🔧 Генерация опций инпута + `removeAttribute('disabled')`.

👁️ Активация плиты управления: `inlinePanel.style.display = 'flex'`.

ФИНАЛ: ВСЕ ВЕЕРНЫЕ РОЛИ ИЗ МАТРИЦЫ ПРАВ ВЫВЕДЕНЫ НА ЭКРАН ОПЕРАТОРА

🔒 Профиль ИБ-защиты подсистемы сбора разрешенных ролей:

- **Защита от подмены лимитов через кэш браузера:** Алгоритм осуществляет извлечение `user_id` текущего оператора панели путем побитового парсинга Payload. Затем он выполняет прямую фильтрацию `usersList.filter` по жестко вычисленному `mySessionUserId`. Попытка подменить текстовый массив ролей в консоли браузера блокируется, так как система пересчитывает лимиты по связи из базы данных.
- **Резервный автономный контур (Air-Gapped Fallback):** Если API бэкенда недоступен из-за сетевого сбоя, логика бесшовно переключается на защищенный ОЗУ-кэш

`window.globalPanelUsersCache`. При этом массив повторно фильтруется строго по совпадению `cleanOpName`, исключая сбой рендеринга.

- **Жесткая ИБ-блокировка при отсутствии прав:** Если у профиля оператора в базе отсутствуют разрешенные системные группы, инпут окрашивается в красный ИБ-маркер `#ef4444`, а кнопка выполнения полностью блокируется, прерывая деструктивный трафик в сеть.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур обеспечивает перехват отправки формы (`secSubmitCustomRoleAssign`), динамическое считывание направления транзакции (`GRANT` или `REVOKE`) и жесткую ИБ-идентификацию оператора для предотвращения анонимных модификаций.

--- Схема верификации сессии и наката/отзыва системных групп ---

ЗАПУСК ТРАНЗАКЦИИ: `secSubmitCustomRoleAssign()`

↓ Считывание направления: `currentAction = actionSelect.value ('GRANT' / 'REVOKE')`

ТРЕХУРОВНЕВЫЙ ИБ-ЗАТВОР ИДЕНТИФИКАЦИИ СЕССИИ ОПЕРАТОРА

Уровень 1: Распил JWT → Уровень 2: Кэш ОЗУ Хрома → Уровень 3: Чтение DOM-шапки

↓ Сверка переменной: `if (!foundUsername || foundUsername === "unknown")`

Имя оператора НЕ определено (Сбой сессии)

- Жесткий ИБ-отказ: Снятие выполнения снайперским затвором.
- 🔔 Вызов `showAlert('Не удалось идентифицировать активную сессию!')`.
- 🔒 Снятие `disabled`-блокировок с главной кнопки.

Имя оператора успешно зафиксировано

- abc Санитизация логина регулярным выражением: `.toLowerCase()`.
- 📦 Упаковка `universalPayload` (Инъект оператора в тело пакета).
- 📡 Трансляция на FastAPI: `secureFetch('/api/manage-system-roles')`.

ФИНАЛ: ВЫЗОВ СЛУЖБЫ ОБНОВЛЕНИЯ РЕЕСТРА АДМИНИСТРАТОРОВ (`loadPanelUsers`)

🛡️ Профиль ИБ-защиты подсистемы наката прав:

- **Жесткий стоп-затвор подмены личности (Anti-Anonymization Barrier):** ИБ-контроллер на строках 463–470 наглухо рубит отправку пакета на бэкэнд, если рантайм не смог динамически верифицировать логин оператора, выполняющего `GRANT/REVOKE`. Это полностью исключает анонимный накат прав в обход сквозного журнала ИБ-аудита (`operator: finalOperator`).
- **Предотвращение зависания кнопок (Anti-Freeze Guard):** Блок выполнения обёрнут в интегрированную секцию `finally`, которая при любом исходе транзакции (успех, крах СУБД,

ошибка сети) гарантированно вызывает `submitBtn.removeAttribute('disabled')`. Это исключает парализацию интерфейса оператора.

КОНТУР ИЗОЛЯЦИИ И УПРАВЛЕНИЯ ПРАВАМИ ИБ (security_rights.js)

Контур обеспечивает изолированное создание учетных записей сотрудников Офицером ИБ с принудительной отправкой паролей на корпоративную почту и жестким маскированием сетевых ошибок.

--- Схема изолированного создания учетных записей сотрудников ---

ЗАПУСК СОЗДАНИЯ РОЛИ: `securitySubmitOnlyCreateUser()`

↓ Валидация полей формы: `if (!server || !username || !email)`

ДЕКОДИРОВАНИЕ ИМЕНИ ОФИЦЕРА ИЗ ПАКЕТА JWT

Изоляция сессии: `currentOp = String(parsed.sub || parsed.username)`

↓ Упаковка неизменяемого затвора: `action_mode = "CREATE_ONLY"`

Ответ: `response.ok == true`

- Вызов `showAlert(..., 'success')`.
- ✏ Мгновенная зачистка текстовых полей формы создания роли сотрудника.

Ответ: `response.ok == false` / Сбой СУБД

- ✗ Перехват сырых технических ошибок СУБД.
- 🔥 Маскирование и вывод абстрактного алерта: 'Отказ СУБД: Сбой'.

ФИНАЛ: ВЫЗОВ `SECTION FINALLY` ДЛЯ СНЯТИЯ `DISABLED-` БЛОКИРОВОК КНОПКИ

🔵 Профиль ИБ-защиты подсистемы изолированного создания УЗ:

- **Жесткое вшивание ИБ-режима (Hardcoded Action Enforcement):** Параметр `action_mode: "CREATE_ONLY"` намертво зашит в JS-код сборщика `payload`-пакета на строках 584–586. Даже если злоумышленник модифицирует HTML-форму на экране, бэкэнд на Python принудительно получит маркер создания, блокируя Офицеру ИБ любую возможность обойти правила Separation of Duties (SoD) и сбросить пароль действующего суперадминистратора.
- **Предотвращение повторных кликов при медленном SMTP-ответе:** Процесс генерации роли и отправки письма на почту может занимать до нескольких секунд. Затвор `submitBtn.setAttribute('disabled', 'true')` на строке 538 полностью исключает риск повторной отправки дублирующих пакетов при сетевых задержках почтового реле-сервера.